How the ML models work

A plain-language guide to what the application learns and how it supports business decisions.

The main job: identify the purchase blocker

GhostDemand converts missed customer enquiries into demand signals. Its main trained model suggests **why a sale was missed**; the business team sees the original message, the organised details and a possible next step.



A rule-based check runs first. A question such as "Do you sell this?" waits in **Needs context** until a missed outcome is established.

1. TF-IDF represents the text

TF-IDF means **term frequency - inverse document frequency**. The app turns 3-5 character fragments into up to 6,000 numeric features. Fragments that distinguish a message from the training collection receive useful weight. For example, "sold" contains "sol" and "old".

2. Logistic regression classifies it

From labeled examples, the model learns weights linking text features to each blocker. It scores all six labels and selects the highest. At the default score threshold of **0.55**, its suggestion can replace the rule-based label; otherwise the app keeps its rule-based fallback.

Six reasons the model can suggest

Out of stock	Price too high	Product not stocked
Quote rejected	Delivery too slow	Enquiry abandoned

EXAMPLE: ONE MESSAGE, TWO KINDS OF WORK

"Need 2 wireless earbuds, but they are sold out. Budget KES 6,000. Needed today."

ML suggests: Out of stock.

Rules extract: Wireless Earbuds; quantity 2; budget KES 6,000; high urgency.

Model scores are not calibrated guarantees of correctness. Automatic captures stay unreviewed until a person checks the blocker.

From learning to action

The model runs on the server. Business rules and human review complete the workflow.

How the demand model is trained

108 synthetic starter examples / 18 per blocker / 6 labels

01 Prepare labeled examples

Combine the bundled practice data with explicitly reviewed customer labels. Duplicate normalised messages count once. Incoming enquiries do not automatically become verified training data.

02 Check the baseline

The trainer uses a stratified 75/25 train-test split and fits evaluation features on the training portion only. Accuracy and macro F1 provide an initial check; synthetic results do not establish real-customer accuracy.

03 Save and serve

The final model refits all labeled examples and saves its vocabulary, TF-IDF values and learned weights as JSON. Flask loads the artifact for backend predictions. Reviewed corrections can enter a later, explicit retraining run.

What learns, and what follows rules?

Purchase blocker	Trained TF-IDF + logistic regression classifier
Product, quantity, budget, urgency	Rules and pattern extraction
Product groups and next steps	Aliases, text similarity and business rules
Unclear enquiry outcomes	Needs context queue and human confirmation

Optional PesaShield SMS model

A separate module uses 2-5 character TF-IDF features and logistic regression to suggest **legit, suspicious or scam**. It uses a trained artifact only when available; below its default 0.60 threshold, or without that artifact, rules take over. PIN-request and fake-reversal rules can override a model result. Its 11 bundled SMS examples are a tiny starter set, and its trainer has no held-out evaluation.

Scope: Automatic channel intake uses local analysis. Product grouping and opportunity priorities are not sales forecasts. Embedding models and voice transcription are not part of the serving demand classifier.

Panel explanation: "TF-IDF turns enquiry text into numbers. Logistic regression suggests the purchase blocker. Rules organise the business details, and people confirm uncertain outcomes."

Implementation: demand_ml.py, fraud_ml.py, app.py, intake.py and intelligence.py.
Inspect the source snapshot on GitHub